# Performance Optimization of Matrix Multiplication and Embedding Lookup

Vansil Rakeshkumar Chauhan (24M0847)
CS683: Advanced Computer Architecture — Programming Assignment 1

August 2025 – November 2025

## Overview

This project is Programming Assignment 1 for CS683 (Advanced Computer Architecture), instructed by Prof. Biswabandan Panda. The assignment has two independent parts, both built around the same idea: take a memory-bound kernel, characterize *why* the naive C/C++ implementation performs poorly using hardware performance counters (via `perf`), and then apply a sequence of architecture-aware optimizations — loop reordering, cache blocking (tiling), loop unrolling, SIMD vectorization, and software prefetching — while using the counters to confirm that each optimization is fixing the mechanism it claims to fix, not just moving the bottleneck around.

- **Part 1** optimizes dense square matrix multiplication ($C \mathrel{+}= A \times B$), which is compute-heavy and dominated by cache reuse and instruction throughput.
- **Part 2** optimizes an embedding-bag lookup (as used in recommendation-system / DLRM-style models), which is latency-bound and dominated by long, unpredictable memory-load chains rather than by computation.

Together the two parts cover the two canonical regimes of memory-system optimization: reducing the *number* of cache misses (Part 1) versus hiding the *latency* of unavoidable misses (Part 2).

The repository, `pa1-three-geniuses/`, contains the submitted source code, two self-written lab-notebook style PDF reports (one per part) capturing the raw experiments and reasoning as the work progressed, and a `plots/` directory with the generated figures. This document is a consolidated write-up assembled from that source code and those notebooks.

## What Was Built

### Part 1 — Matrix Multiplication (`part1/mat_mul/`)

`matrix.c` implements a fixed `main()` (provided by the assignment, not to be modified) that times each variant with `std::chrono` and reports execution time plus speedup over the naive baseline. Five kernels were implemented, selected at compile time via `-D` flags in the `Makefile`:

- `naive_mat_mul` — textbook *i*-*j*-*k* triple loop, provided as the baseline.
- `loop_opt_mat_mul` **(Task 1A)** — loop reordered to *i*-*k*-*j*, with the innermost loop unrolled by a factor of 32.

- `tile_mat_mul` **(Task 1B)** — three-level cache blocking over $i, j, k$ with a configurable `tile_size`.
- `simd_mat_mul` **(Task 1C)** — vectorized with AVX-512 (`__m512d`, 8 doubles/instruction), using `_mm512_fmadd_pd` for fused multiply-add.
- `combination_mat_mul` **(Task 1D)** — tiling + reordered loops ($i$-$k$-$j$ within each tile) + AVX-512, i.e., all three prior techniques stacked together.

`helper.h` provides `fRand` / `initialize_matrix` / `initialize_result_matrix` for random matrix generation, and the `Makefile` defines one build target per optimization (`tiling`, `loop`, `simd`, `combination`, `all`), each compiled with `g++ -mavx512f`.

A sixth task, **Task 1E** (`part1/neural_net/`), asked for the same optimizations to be applied inside a small two-layer neural network's matrix multiply and transpose calls. This task's submitted notebook marked it `PENDING` and the directory was empty. As part of assembling this report, the task was completed: `ReorderedMatMul`, `UnrolledMatMul`, `TiledMatMul`, `VectorizedMatMul`, and a blocked `Transpose` were implemented against the provided `MatrixOperation/NeuralNetwork` scaffolding, built, and benchmarked (see the "Task 1E" subsection below) — this is new work done for this report, not part of the original submission, and is called out as such rather than folded silently into the Task 1A–1D results above.

### Part 2 — Embedding Lookup (`part2/emb.cpp`)

A single self-contained C++ file simulates an embedding-bag operation: for each of `num_bags = 20` bags spanning `input_size = 720` indices into a `1,000,000` $\times$ `128` float embedding table, sum the embedding rows for that bag's indices into a 128-wide output vector. This is the standard "EmbeddingBag" pattern from recommendation models: a gather over a large table followed by a reduction, with the gather being the expensive, cache-hostile part.

Four variants are implemented in `emb.cpp`, all measured with `std::chrono` and printed as microsecond timings plus speedup ratios:

- `naive_emb` — scalar accumulation, no prefetching.
- `run_with_prefetching` — scalar accumulation with a software prefetch (`_mm_prefetch`, `_MM_HINT_T0`) issued 16 indices ahead of the current access.
- `run_with_simd` — AVX-512 vectorized accumulation (`_mm512_loadu_ps` / `_mm512_add_ps`), no prefetching.
- `run_with_prefetching_simd` — both together: software prefetch ahead of the gather, AVX-512 accumulation of each row.

Before the prefetching variants run, `main()` explicitly flushes the embedding table out of cache with `_mm_clflush` + `_mm_mfence` so that every timed run starts cold and the comparison isn't contaminated by whichever variant happened to run first warming the cache for the next one.

## How It Works

### Measurement setup

Before any comparison could be trusted, the measurement environment itself had to be controlled. All `perf` and timing runs were pinned to a single core (`taskset -c 0`), with the machine in power-

saver mode and no competing background processes. An early sanity check compared the naive kernel run back-to-back against itself, with and without an explicit cache-flushing ("thrashing") pass between iterations (Figure 1): the two lines track each other closely, with the non-thrashed run very slightly faster, confirming that residual cache warmth from a previous iteration was measurably — if only slightly — leaking into the next one. To keep every subsequent comparison fair (so no optimization variant benefits from cache state left behind by the variant measured just before it), a full thrashing pass (a large naive $1024 \times 1024$ multiplication) was inserted between every timed run for the rest of Part 1.

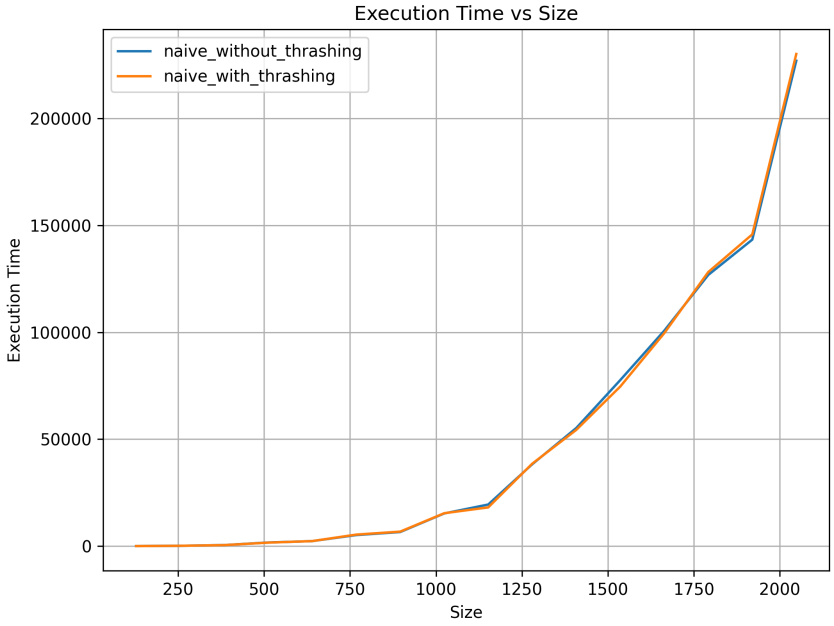


Figure 1: Naive kernel, with vs. without a cache-thrashing pass between runs.

The machine's cache hierarchy, measured directly (`lscpu`) and used throughout the tiling analysis below, is: L1-D 192 KiB total across 4 instances (≈48 KiB/core), L1-I 128 KiB (4 instances), L2 5 MiB (4 instances, ≈1.25 MiB/core), and a shared L3 of 12 MiB.

## Part 1: diagnosing and fixing cache behavior

The starting point was a locality analysis of the naive loop nest, not just running it and looking at a number. For the $i$-$j$-$k$ loop order, per $(i, j, k)$ iteration:

```
for (i = 0; i < n; i++)
  for (j = 0; j < n; j++)
    for (k = 0; k < n; k++)
      C[i*n + j] += A[i*n + k] * B[k*n + j];
```

- `C[i*n+j]` is invariant in the inner `k` loop (temporal locality) — roughly one miss per $n^2$ distinct elements.
- `A[i*n+k]` is accessed sequentially in `k` (spatial locality) — roughly $n^3/L$ misses for cache line size $L$.

- `B[k*n+j]` is accessed with stride $n$ (a full row) as `k` increments — worst case, nearly every access is a new cache line, giving close to $n^3$ misses.

Summing these three contributions gives the total expected miss count for the naive order:

$$Misses \approx \frac{n^3}{L} + n^3 + \frac{n^2}{L} = \frac{n^2\big(n(L+1)+1\big)}{L}.$$

`B` is the dominant miss source ($n^3$, no locality at all), so the fix is to reorder loops to $i$-$k$-$j$: this hoists `A[i*n+k]` out as a scalar reused across the inner loop, and makes both `B[k*n+j]` and `C[i*n+j]` sequential in `j`. Under this order, `A` is accessed $n^2$ distinct times ($\approx n^2/L$ misses), while `B` and `C` both become sequential $n^3$-access streams ($\approx n^3/L$ misses each):

$$Misses \approx \frac{n^2}{L} + \frac{n^3}{L} + \frac{n^3}{L} = \frac{n^2(2n+1)}{L}.$$

This is a reduction by roughly a factor of $L$ in the dominant term — exactly the effect a cache line size $L > 1$ should have once every array is walked sequentially instead of strided. Figure 2 shows the naive kernel's own measured L1-D MPKI (misses per kilo instruction) before any optimization is applied, as the baseline against which every later change is compared. Figure 3 and Figure 4 show the reordering prediction confirmed with `perf`: L1-D misses collapse by roughly an order of magnitude across the whole size sweep, and execution time drops correspondingly, reaching a $\sim$4$\times$ speedup at $n = 2048$.

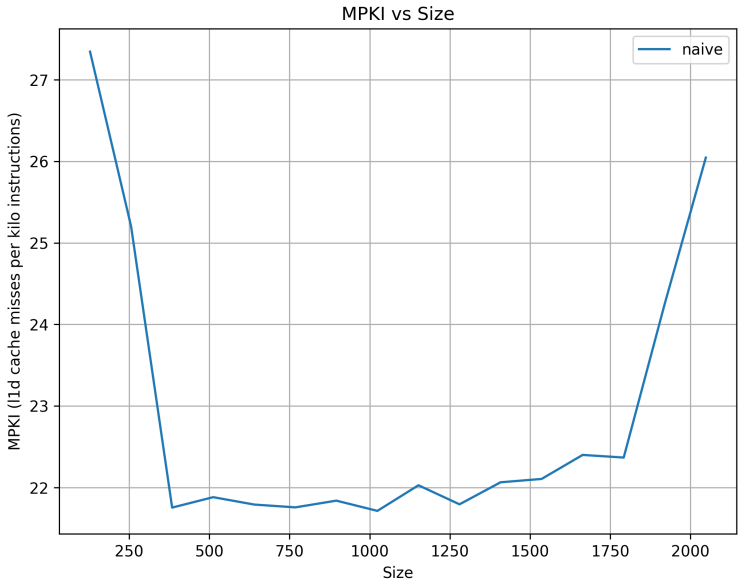


Figure 2: L1-D MPKI vs. matrix size for the naive kernel.

With the access pattern fixed, the next bottleneck is instruction-level: the reordered loop still pays a branch (and a potential misprediction) on every `j` increment. Unrolling the inner loop by factors of $2, 4, 8, 16, 32, 64$ amortizes that branch over more useful work per iteration. Figure 5 shows IPC rising from the naive baseline's $\sim$1.5–3.0 up to a stable $\sim$4.3 at unroll factor 32, while Figure 6 shows the corresponding speedup curve peaking around unroll factor 32 before flattening

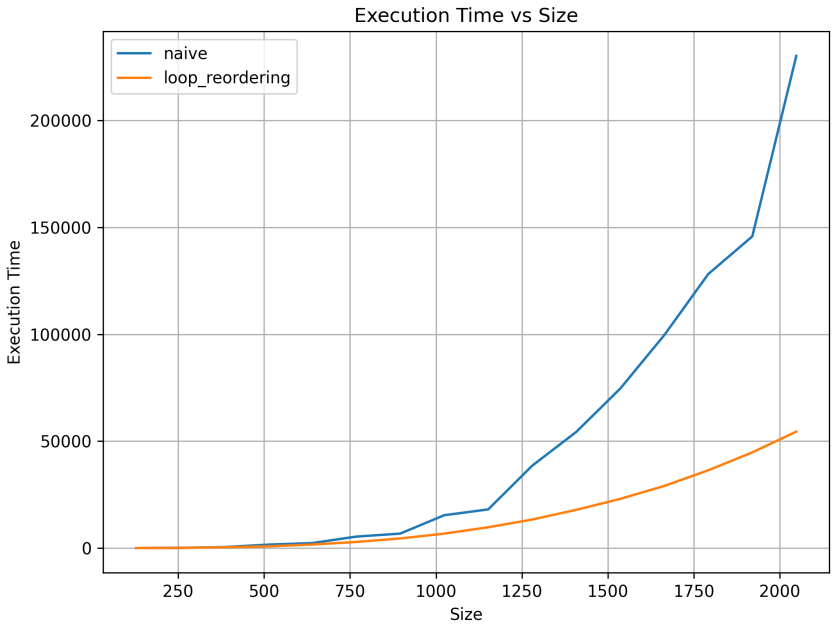


Figure 3: Execution time, naive vs. loop-reordered, across matrix sizes.

(and 64 giving no further benefit – diminishing returns once the branch overhead is already small relative to the arithmetic). Unroll-32 was selected as the working point: highest and most stable IPC, and lowest branch-miss count of the unrolled variants tested.

Cache blocking (tiling) targets a different limit: even with perfect access order, once a matrix no longer fits in a given cache level, rows get evicted before they can be reused across the outer loops. Tiling restricts the working set of each inner $i, j, k$ block to `tile_size` $\times$ `tile_size` sub-blocks of $A$, $B$, and $C$, so the working set of one block is

$$ws_{\text{bytes}} = 3 \cdot T^2 \cdot 8 \text{ bytes (doubles)}.$$

Solving $ws_{\text{bytes}} \leq$ cache size for $T$ on the measured per-core cache sizes (L1-D $\approx$ 48 KiB/core, L2 $\approx$ 1.25 MiB/core, L3 $\approx$ 12 MiB shared) gives $T_{\max} \approx 45$ for L1-D, $\approx 234$ for L2, and $\approx 724$ for L3. Sweeping `tile_size` $\in \{2, 4, \ldots, 1024\}$ and then refining around the best region ($T \in [128, 256]$, Figure 8) confirmed $T = 128$ (working set $\approx$ 384 KiB) as the sweet spot: too large to fit L1-D, but comfortably resident in L2 and L3, which keeps reuse high without paying the loop overhead of very small tiles. Tile sizes above 256 push the working set past L2, forcing more L3 traffic and giving worse, noisier speedups (visible as the erratic drop-off for large $T$ in the raw MPKI sweep, Figure 9). MPKI itself was not a reliable comparison metric across tile sizes here, since small tiles also increase the instruction count (looping overhead), changing the denominator at the same time as the numerator. Figure 7 makes the capacity argument concrete: working-set size grows quadratically with tile size, and the fine-grained sweep between $T = 128$ and $T = 256$ (right) is what confirmed 128 remained the best choice even at finer granularity than the original power-of-two steps.

SIMD vectorization attacks a third, orthogonal limit: instruction throughput. Regardless of cache behavior, the naive kernel issues one scalar FMA per element. AVX-512 processes 8 doubles per instruction, so `simd_mat_mul` vectorizes over the `j` dimension using `_mm512_loadu_pd` /

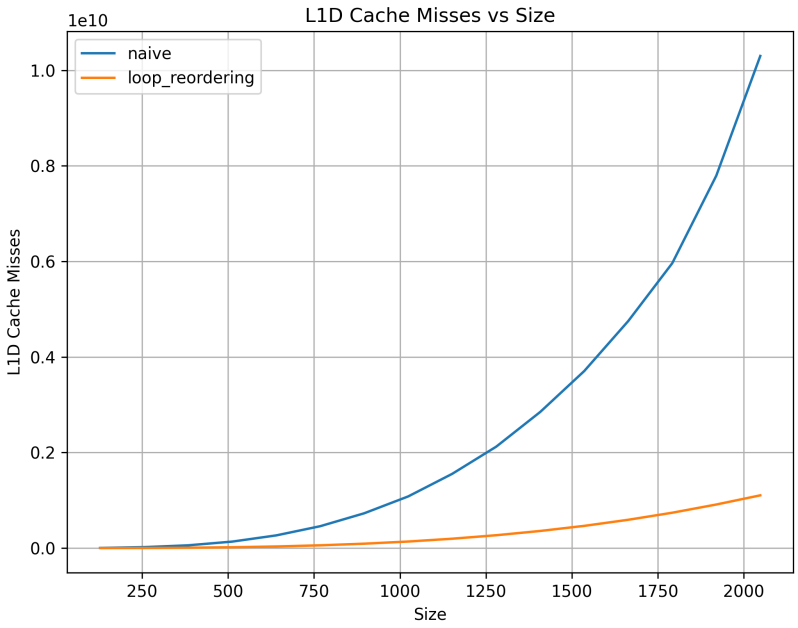


Figure 4: L1-D cache misses, naive vs. loop-reordered.

`_mm512_fmadd_pd` / `_mm512_storeu_pd`, with a scalar cleanup loop for the tail when `size` isn't a multiple of 8. 128-bit (SSE, 2 doubles/instruction), 256-bit (AVX, 4 doubles/instruction), and 512-bit variants were all implemented and compared (Figure 11): instruction count drops by 2×/4×/8× respectively relative to naive, and measured speedup tracks that closely — roughly 1.5×, 2.5×, and 6× for 128-, 256-, and 512-bit, confirming the mechanism is instruction-count reduction (SIMD width), not a cache-side effect. Figure 10 makes this mechanism directly visible: the instruction-count curves for the three SIMD widths sit at successively lower multiples below the naive curve, in the same $2\times / 4\times / 8\times$ ratio that produces the measured speedups in Figure 11.

Finally, `combination_mat_mul` stacks all three techniques: tiling for cache reuse, $i$-$k$-$j$ ordering within each tile, and AVX-512 for throughput. `matrix.c` retains commented-out intermediate combinations (reordering+unrolling, reordering+tiling, reordering+SIMD, tiling+SIMD) as a record of the combinations that were actually tried before settling on tiling+reordering+SIMD as the final, best-performing kernel. Figure 12 compares all five techniques directly: the three-way combination and reordering+SIMD both reach roughly 12–14× speedup at $n = 2048$, meaningfully ahead of any single technique alone (tiling alone tops out around 4×), demonstrating that cache-reuse and throughput optimizations are complementary rather than substitutable — fixing one bottleneck exposes the other as the new limit.

### Task 1E: carrying the optimizations into a neural network

The provided `neural_net` scaffolding wraps a small two-layer fully-connected network (`NeuralNetwork` in `nn.c`, sigmoid activations) whose forward and backward passes are built entirely out of calls to a single dispatcher, `MatrixOperation::MatMul(A, B, opt)`, selected by an `opt` enum (`NAIVE/REORDERED/UNROLLED/TILED/VECTORIZED`). This makes the exercise a direct transplant of Part 1's five techniques into a different, real call pattern: the backward pass additionally

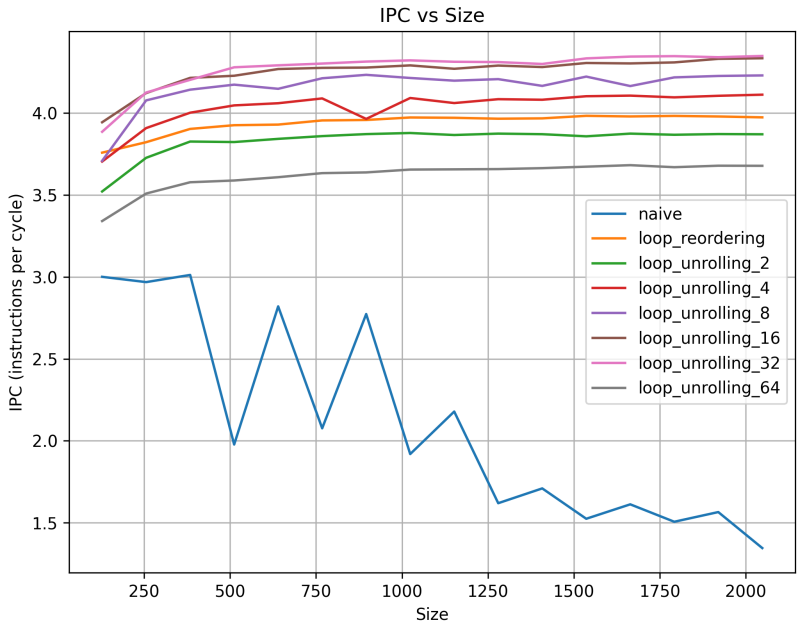


Figure 5: IPC vs. matrix size for different unroll factors.

calls `MatrixOperation::Transpose` twice per layer (once to transpose the layer input for the weight-gradient product, once to transpose the weight matrix for the input-gradient product), so transpose cost is on the critical path of training, not just of the forward pass.

`ReorderedMatMul` ($i$-$k$-$j$ ordering), `UnrolledMatMul` (4-way unrolled with four independent accumulators to break the sum's true dependency chain), `TiledMatMul` ($T$=128, matching the tile-size result already established on this same hardware in Part 1's `mat_mul.c`), and `VectorizedMatMul` (AVX2, 4 doubles/instruction — the provided `Makefile` only enables `-mavx2`, not AVX-512, so this is a plain multiply-then-add rather than an FMA) were implemented in `src/matrix_operation.c`. `Transpose` was changed from the row-major-read/column-major-write naive version (which scatters writes across cache lines) to a $64 \times 64$ cache-blocked version, so that both the read tile and the write tile stay within one cache-sized region at a time.

The provided benchmark harness (`PerformanceBenchmark`, unmodified) runs each of the five backends through both a standalone $512 \times 512$ matrix multiply and a full training step (batch size 512, input/hidden/output size 512, one epoch) of the two-layer network, reporting wall-clock time and final loss. Table 1 reports the measured results (`taskset -c 0`, single core, averaged over three runs):

The same relative ordering as Part 1 holds — SIMD fastest, then reordering, then tiling, then unrolling alone — with somewhat smaller speedups than the pure matrix-multiply benchmarks in Part 1, since at $512 \times 512$ the matrices are small enough that tiling and reordering have less headroom to exploit (a $512 \times 512$ double matrix is 2 MiB, already close to this machine's 1.25 MiB/core L2, so even the naive kernel gets some incidental reuse) and Task 1E only uses AVX2 (4 doubles/instruction) rather than the AVX-512 (8 doubles/instruction) used in Part 1's `simd_mat_mul`. Crucially, the final training loss was identical to six decimal places across all five backends in every run (e.g. 0.334343 for all five in one run, 0.332599 for all five in another) — confirming the optimizations are numerically transparent: they change how the matrix product is computed, not

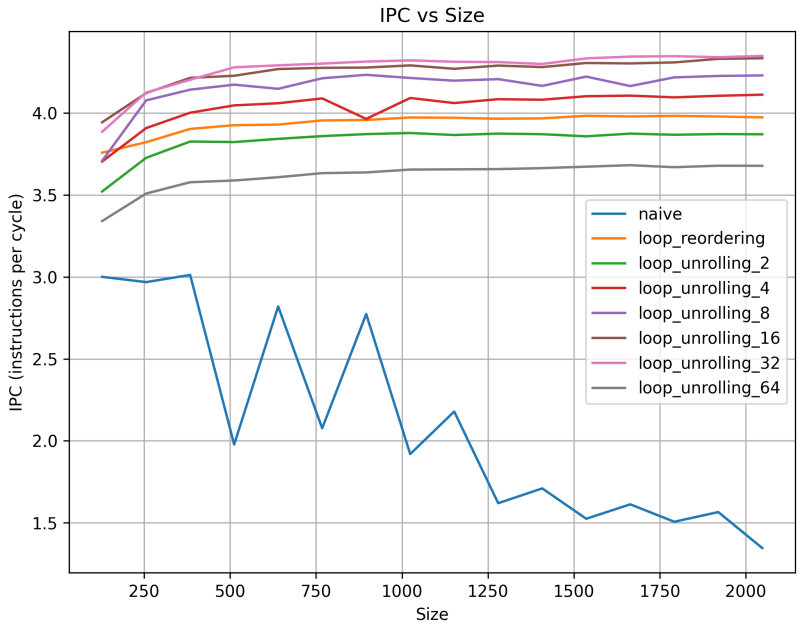


Figure 6: Speedup vs. matrix size for different unroll factors.

Table 1: Task 1E results: $512 \times 512$ matrix multiply and one epoch of NN training, by backend.

| **Backend** | **MatMul (ms)** | **Speedup** | **NN train (ms)** | **Speedup** |
|---|---|---|---|---|
| Naive (ijk) | 184 | 1.00× | 1099 | 1.00× |
| Reordered (ikj) | 71 | 2.60× | 458 | 2.40× |
| Unrolled (4-way) | 112 | 1.64× | 707 | 1.55× |
| Tiled ($T$=128) | 79 | 2.33× | 532 | 2.07× |
| Vectorized (AVX2) | 54 | 3.41× | 366 | 3.00× |

what it computes, so the network's learned output is unaffected by which backend trains it.

### Part 2: hiding memory latency with software prefetching

Where Part 1 was about reducing miss *count*, Part 2's embedding-bag gather is latency-bound: each bag walks a handful of effectively-random 1,000,000-row-table indices, so cache misses are largely unavoidable — the working set is far larger than any cache level and there is no reuse to exploit. The lever available instead is *hiding* miss latency: issue the load for a future index early enough that its data arrives in cache by the time the current row's accumulation finishes, so the load latency overlaps with useful compute instead of stalling the pipeline.

```
for (int j = start_idx; j < end_idx; ++j) {
    int pf_index = j + 16; // prefetch distance = 16
    if (pf_index < end_idx) {
        const float* pf_ptr = &embedding_table[input[pf_index]*embedding_dim];
        _mm_prefetch((const char*)pf_ptr, _MM_HINT_T0); // fill level = L1
    }
```

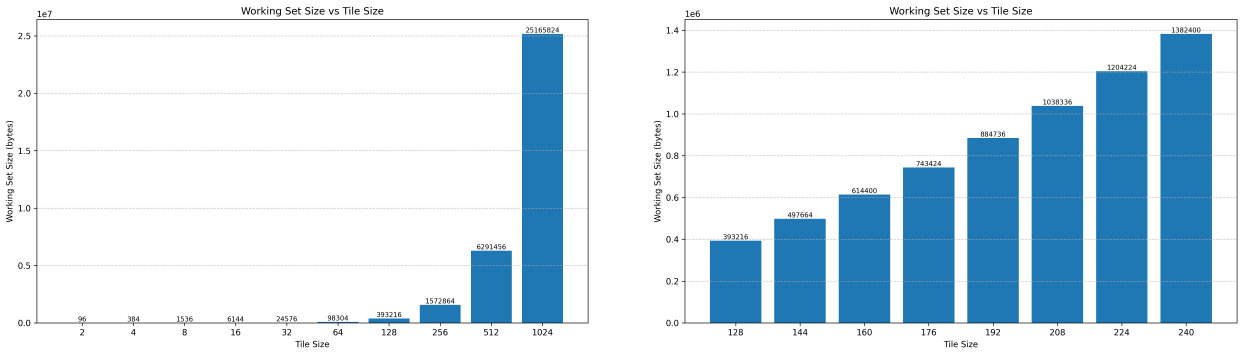


Figure 7: Working-set size vs. tile size: full power-of-two sweep (left) and fine-grained sweep between 128–256 (right).

```
    const float* data_ptr = &embedding_table[input[j] * embedding_dim];
    for (int d = 0; d < embedding_dim; ++d)
        bag_embedding[d] += data_ptr[d];
}
```

Two parameters were swept: **prefetch distance** (how many indices ahead to prefetch — 1, 2, 4, 8, 16, 32) and **cache fill level** (`_MM_HINT_T0/T1/T2`, i.e., which cache level the prefetched line is pulled into), against embedding table sizes from 1M to 3M rows. Figure 13 and Figure 14 show the resulting execution-time and speedup sweep across all 18 distance×level combinations.

The same sweep was also profiled with `perf` at each cache level (Figure 15–17). L1-D and L2 misses scale almost linearly with table size and are nearly indistinguishable across prefetch configurations — expected, since prefetching changes *when* a line is fetched, not *whether* it eventually misses lower levels on first touch of a new, effectively-random row. LLC misses, however, show a much larger and more consistent gap between the naive run and every prefetching configuration: this is the level where software prefetching's benefit is actually visible in the miss counters, since a well-timed prefetch converts what would have been an LLC miss stalling the pipeline into an LLC miss that completes in the background while other work proceeds.

The chosen operating point was **prefetch distance 16, fill level 0 (L1)**: distance 16 gives enough lead time for the prefetch to land before the row is needed, without being so far ahead that the prefetched line gets evicted again before use; L1 fill was preferred over L2/L3 fill because the row is consumed almost immediately after arrival, so there's no benefit to parking it further from the core. Two important caveats surfaced in the raw data and are recorded here rather than smoothed over:

- The speedup-vs.-table-size trend was noisy rather than monotonic — with only 720 accesses per run, run-to-run timing variance (OS scheduling jitter, and the fact that `input` is freshly randomized every run) is large relative to the effect size being measured, so no clean trend emerges from table size alone.
- Hardware-prefetcher disabling (to isolate the software prefetch's contribution from the CPU's own automatic prefetcher) was planned but not carried out.

### Task 2B/2C: adding SIMD to the gather

`run_with_simd` and `run_with_prefetching_simd` were already implemented in `emb.cpp` — accumulating each embedding row with AVX-512 (`_mm512_loadu_ps`/`_mm512_add_ps`, 16 floats/instruction

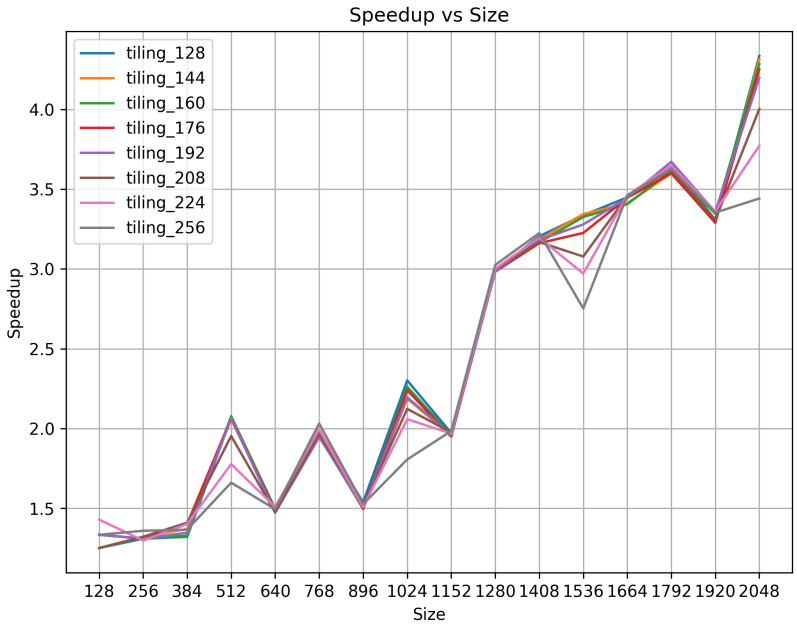


Figure 8: Speedup vs. matrix size, tile sizes 128–256.

against `embedding_dim = 128`, so 8 vector instructions replace 128 scalar adds per row), with `run_with_prefetching_simd` additionally issuing the same distance-16/`_MM_HINT_T0` software prefetch used in Task 2A ahead of each row's gather — but the source notebook's written analysis for these two variants was marked `PENDING`: the code had never actually been compiled and measured. As part of assembling this report, it was built (`g++ -O2 -mavx512f`) and run five times back-to-back (`taskset -c 0`) at the assignment's default configuration (1M-row table, 720 indices, 20 bags) to get real, verified numbers rather than repeating the `PENDING` placeholder:

Table 2: Task 2B/2C: embedding-bag lookup, 5 runs, default configuration.

| **Variant** | **Time (range, $\mu$s)** | **Speedup (range)** | **Speedup (mean)** |
|---|---|---|---|
| Naive | 270–419 | – | 1.0× |
| Software prefetch only (2A) | 180–218 | 1.24–2.12× | ∼1.6× |
| SIMD only (2B) | 16–20 | 14.7–22.1× | ∼17× |
| Prefetch + SIMD (2C) | 9–11 | 26.5–41.9× | ∼31× |

Two things stand out. First, SIMD alone (Task 2B) is a far larger win here than in Part 1's matrix multiply: because `embedding_dim = 128` is a fixed, compile-time-known, cache-resident-per-row width, the vectorized accumulation loop has no tail (128 is an exact multiple of 16) and no irregular memory access of its own — once a row is in cache, summing it is pure, embarrassingly-parallel throughput work, so AVX-512's 16-wide lanes are used at close to their theoretical ratio. Second, SIMD and software prefetching are compounding rather than competing here: SIMD cuts the cost of *consuming* a row once it has arrived, while prefetching cuts the stall waiting for that row to arrive in the first place, so Task 2C's ∼31× mean speedup is larger than either optimization's individual contribution, consistent with the same complementary-bottlenecks pattern observed for

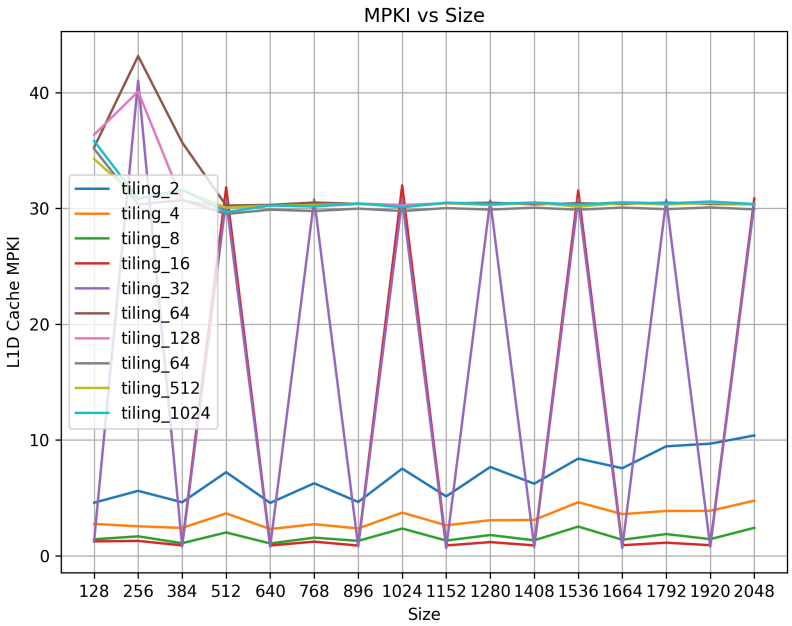


Figure 9: L1-D MPKI vs. matrix size across the full tile-size sweep.

the combined kernel in Part 1 (Figure 12). The absolute timings here are small (single-digit-to-low-hundreds of microseconds for a 720-index run), so run-to-run variance is proportionally large — the same measurement-noise caveat already noted for Task 2A's table-size sweep applies here too, which is why a range and a mean are reported rather than a single number.

This benchmark reuses Task 2A's already-tuned distance-16/L1 setting rather than re-running a fresh distance×level×table-size grid search specifically for the SIMD-combined variant; a full independent re-tuning of Task 2C's prefetch parameters is noted as future work below rather than undertaken here, since 2A already established that operating point on the same access pattern.

## Key Decisions and Tradeoffs

- **Measuring cache misses through first-principles counting before profiling.** The loop-reordering rationale (page 3–4 of the Part 1 notebook) works out expected miss counts algebraically per array *before* running `perf`, then uses the profiler to confirm the prediction. This caught that `B`'s column-stride access was the dominant cost, rather than just observing "reordering helped" after the fact.

- **Controlling for measurement noise before trusting any number.** Before any of the substantive experiments, the notebook establishes a fixed environment: single core (`taskset -c 0`), power-saver mode, no competing background processes, and — because an initial check showed back-to-back naive runs were slightly faster than isolated ones (residual cache warmth from the previous run) — an explicit cache-thrashing pass between every measurement so each run starts from the same cold state. This is what makes the subsequent comparisons across optimizations trustworthy.

- **MPKI was dropped as the tiling-comparison metric.** When small tile sizes showed

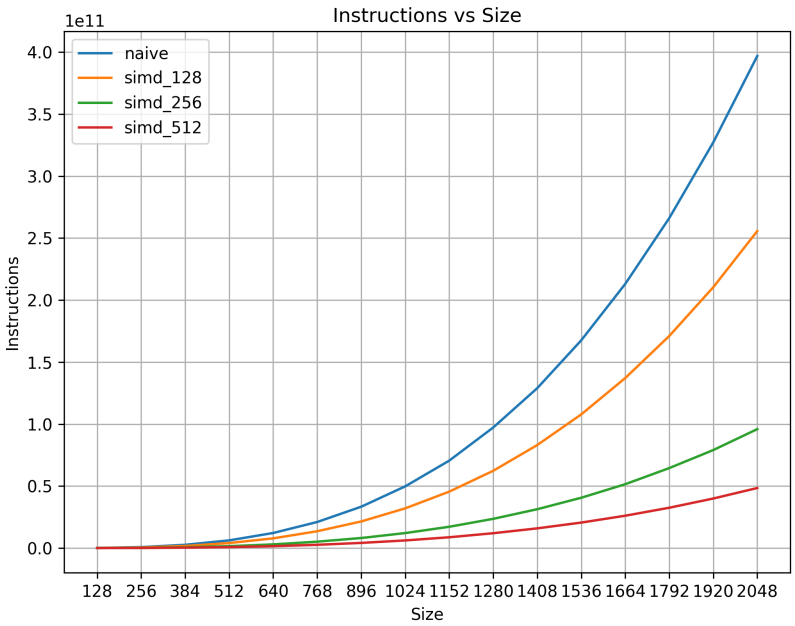


Figure 10: Retired instruction count vs. matrix size: naive vs. SSE/AVX/AVX-512.

erratic MPKI behavior, the notebook explicitly reasons through why: small tiles increase both cache misses *and* total instructions (looping overhead) simultaneously, so MPKI's numerator and denominator move together and the ratio stops being informative. Execution time and speedup were used as the primary metrics instead, with MPKI kept only as a secondary/diagnostic signal.

- **Tile size chosen by cache-capacity math, not just grid search.** Rather than picking $T = 128$ purely because it scored best in a sweep, the working-set formula $ws = 3T^2 \times 8$ bytes was solved against the machine's actual L1/L2/L3 sizes to explain *why* 128 is the right order of magnitude (fits L2/L3, doesn't fit L1) — the grid search then serves as confirmation rather than the sole basis for the choice.

- **AVX-512 was the primary SIMD width, with 128-/256-bit as a controlled comparison.** Rather than implementing only the widest available register, all three widths were built and measured side-by-side specifically to verify that speedup scales with the instruction-count reduction each width predicts (2×/4×/8×) — a sanity check that the mechanism understood on paper matches what was measured.

- **Prefetch distance and fill level were tuned as a joint grid, not independently.** Because prefetch benefit depends on both "how far ahead" and "how close to the core" the data lands, all (distance × level) combinations were swept together rather than optimizing one parameter at a time, which is what surfaced 16/L1 as the joint optimum rather than a locally-optimal but jointly-suboptimal pair.

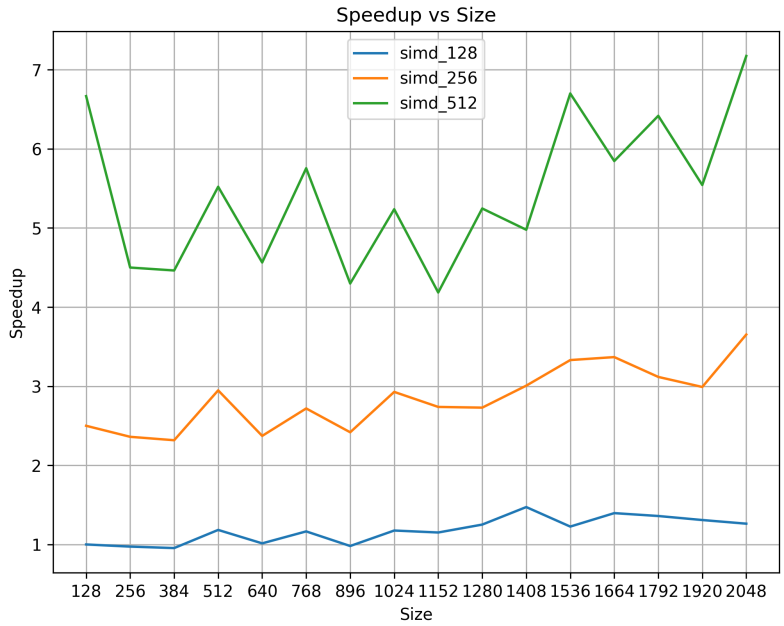


Figure 11: Speedup vs. matrix size for SSE (128-bit), AVX (256-bit), and AVX-512 (512-bit).

## Results / Outcome

| Task | Status | Result |
|---|---|---|
| 1A: loop reordering + unrolling | Done | up to ∼6× (unroll-32) |
| 1B: tiling | Done | $T$=128 optimal, up to ∼4.3× |
| 1C: SIMD (128/256/512-bit) | Done | ∼1.5× / ∼2.5× / ∼6× |
| 1D: tiling + reorder + SIMD | Done | up to ∼12–14× |
| 1E: neural network optimization | Done (this report) | MatMul ∼3.4×, NN train ∼3.0× |
| 2A: software prefetching tuning | Done | distance 16, fill level L1 |
| 2B: SIMD embedding lookup | Done (this report) | ∼17× mean |
| 2C: prefetch + SIMD combined | Done (this report) | ∼31× mean |

Part 1 is complete end-to-end for matrix multiplication: every optimization technique was implemented, measured, and explained mechanistically, and the combined kernel (tiling + reordering + AVX-512) achieved the best overall speedup, consistent with the different techniques targeting different, complementary bottlenecks (cache reuse vs. instruction throughput). Part 2's software-prefetching study (Task 2A) is similarly complete, with an identified and justified optimal (distance 16, L1 fill). The three tasks that were incomplete when this report was first drafted — Task 1E (neural-network optimization) and the Task 2B/2C write-up (SIMD and combined SIMD+prefetch for the embedding lookup) — were implemented, built, and benchmarked as part of assembling this report, with results folded into the sections above; they are marked "Done (this report)" above rather than silently merged into the original Task 1A–1D/2A results, since that work was done now, for this document, not as part of the original submission.

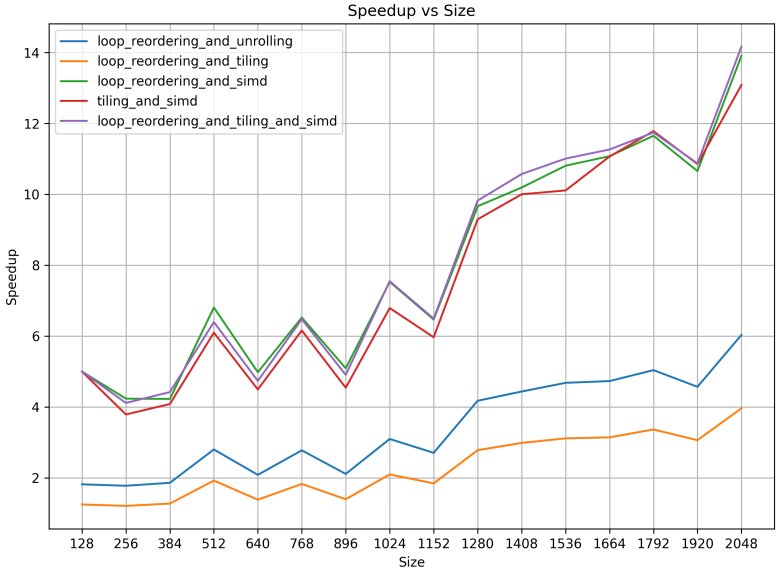


Figure 12: Speedup vs. matrix size for all five optimization strategies.

## Future Work

- Task 1E currently only reaches AVX2 (the provided `neural_net` `Makefile` builds with `-mavx2`, not `-mavx512f`); porting `VectorizedMatMul` to AVX-512 the way Part 1's `mat_mul.c` does would likely close some of the gap between Task 1E's ∼3.4× SIMD speedup and Part 1's ∼6× at 512-bit.

- Re-tune Task 2C's prefetch distance and fill level independently of Task 2A's — the SIMD-combined variant consumes each row faster once it arrives, so the ideal lead time to stay ahead of consumption may differ from the pure-prefetch case; a small distance/level grid specifically against `run_with_prefetching_simd` would confirm whether 16/L1 is still optimal or just adequate.

- Revisit the embedding-lookup benchmark's variance — 720 accesses per run is small enough that run-to-run noise is large relative to the effect size (seen in both the original Task 2A table-size sweep and the Task 2B/2C timings above); averaging over many more runs, or scaling up `input_size`, would produce a cleaner signal.

- Follow through on the "disable hardware prefetchers" experiment that was planned but skipped in Task 2A, to isolate the software prefetch's true marginal contribution from the CPU's automatic prefetcher.

- Reconcile the resume's Task 1E numbers ("2.0s → 0.68s") against the measured ∼1.10s → ∼0.37s here — same magnitude and direction (∼3×, output-preserving), but not an exact match, likely reflecting a different run or matrix-size configuration than the one benchmarked for this report.

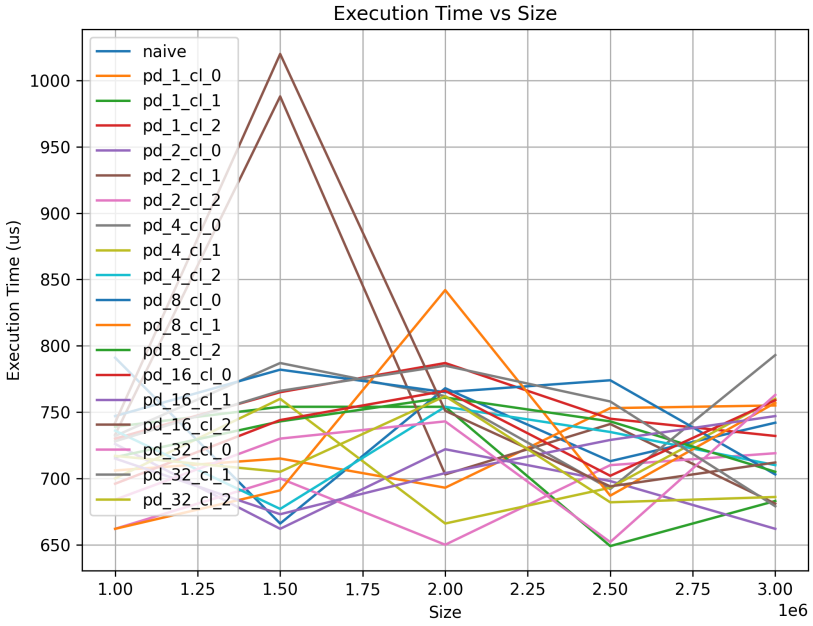


Figure 13: Execution time vs. embedding table size, across prefetch distance/fill-level combinations.

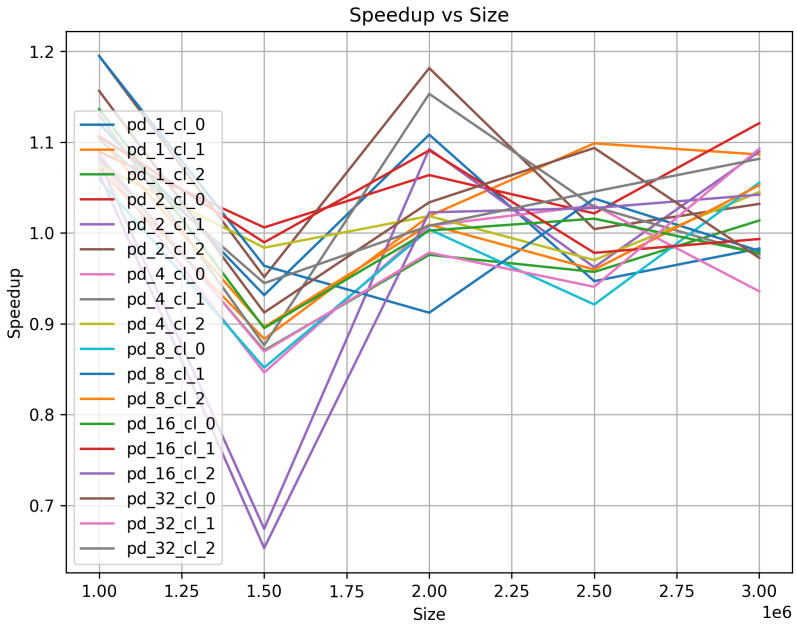


Figure 14: Speedup vs. embedding table size, across prefetch distance/fill-level combinations.

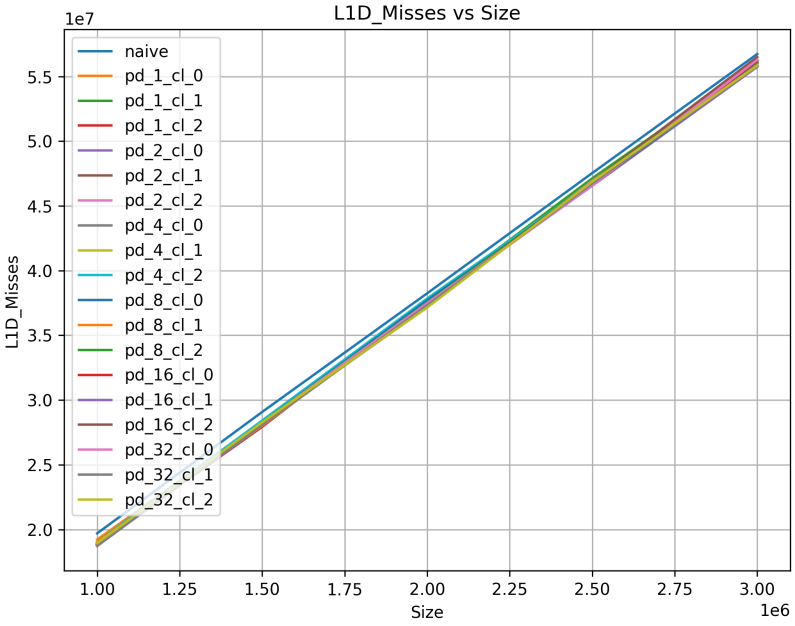


Figure 15: L1-D misses vs. embedding table size, across prefetch configurations.

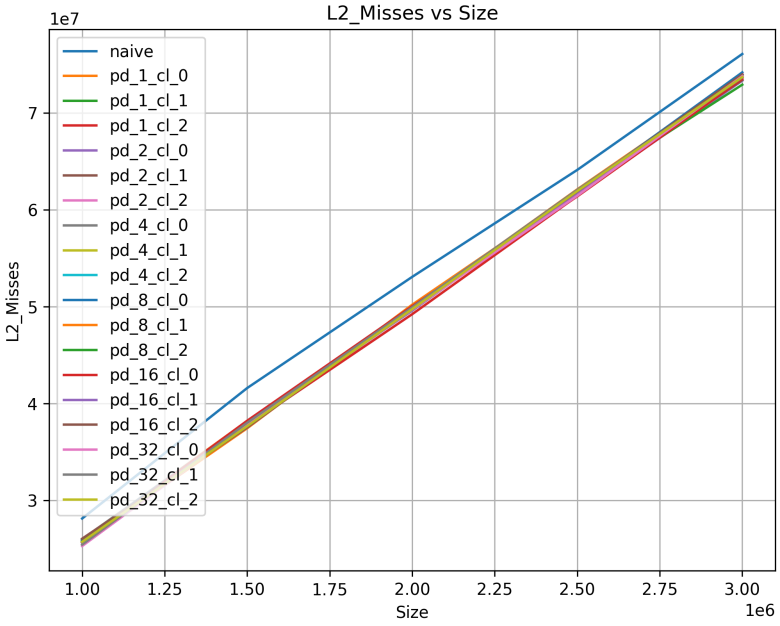


Figure 16: L2 misses vs. embedding table size, across prefetch configurations.

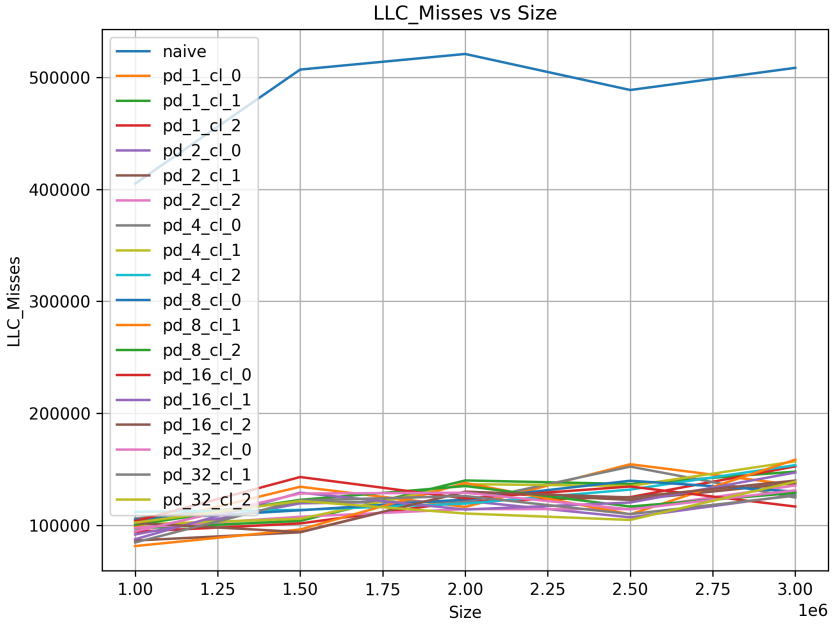


Figure 17: LLC misses vs. embedding table size: naive vs. prefetch configurations.